

NONGSHIM USA

# Runtime User Manual

AI Vision Inspection System

---

Version 1.0

Revised: May 2026

For internal use only

CONFIDENTIAL



# Table of Contents

---

|          |                                    |           |
|----------|------------------------------------|-----------|
| <b>1</b> | <b>System Overview</b>             | <b>3</b>  |
| 1.1      | What the system does               |           |
| 1.2      | Key components                     |           |
| <b>2</b> | <b>First Launch</b>                | <b>4</b>  |
| 2.1      | Starting the application           |           |
| 2.2      | Accessing the dashboard            |           |
| 2.3      | Stopping the application           |           |
| <b>3</b> | <b>Dashboard Overview</b>          | <b>5</b>  |
| 3.1      | Camera cards                       |           |
| 3.2      | Per-camera controls                |           |
| 3.3      | Dashboard layout                   |           |
| <b>4</b> | <b>Line (Camera) Configuration</b> | <b>6</b>  |
| 4.1      | Adding a new line                  |           |
| 4.2      | Camera settings                    |           |
| 4.3      | Detector type                      |           |
| 4.4      | Confidence thresholds              |           |
| <b>5</b> | <b>Save Path Configuration</b>     | <b>9</b>  |
| 5.1      | Where images are saved             |           |
| 5.2      | Setting the save path              |           |
| 5.3      | Saved folder structure             |           |
| 5.4      | AWS S3 remote storage              |           |
| <b>6</b> | <b>Settings Files Reference</b>    | <b>11</b> |
| 6.1      | config.json (per-line)             |           |
| 6.2      | global_settings.json               |           |
| 6.3      | Path conventions                   |           |
| <b>7</b> | <b>Admin Page</b>                  | <b>14</b> |
| 7.1      | Accessing admin                    |           |

7.2 Global settings

7.3 S3 configuration

**8 Troubleshooting** **15**

# 1 System Overview

## 1.1 What the system does

NS Runtime is a real-time AI vision inspection platform. It connects to up to **10 cameras** on production lines, runs an AI model on every frame, and automatically triggers a reject signal when a defective product is detected. All defect images are saved to disk (and optionally to AWS S3) for traceability.

### Quick summary

Camera feed → AI detection → Reject signal → Image saved to disk

## 1.2 Key components

| Component                | Description                                                                                              |
|--------------------------|----------------------------------------------------------------------------------------------------------|
| <b>Dashboard</b>         | Web UI showing live camera feeds and controls. Open in a browser at <code>http://localhost:5173</code> . |
| <b>Backend API</b>       | FastAPI server (port 8000) that manages workers, streams video, and serves settings.                     |
| <b>Inspection Worker</b> | Background thread per camera. Runs AI inference and triggers the reject signal.                          |
| <b>Detector</b>          | Pluggable AI model: YOLO (object detection), PaddleOCR (text), or CNN (classification).                  |
| <b>Rejecter</b>          | Sends a timed I/O signal to the conveyor actuator after a configurable frame delay.                      |
| <b>Data Manager</b>      | Saves defect images, maintains a preview folder, and auto-archives old files.                            |

## 1.3 Supported camera types

| Camera                   | How to connect               | Notes                                                         |
|--------------------------|------------------------------|---------------------------------------------------------------|
| Basler GigE (aca series) | Ethernet (same subnet as PC) | Highest performance; requires <code>.pfs</code> settings file |
| USB Webcam               | USB port                     | Easy setup; lower frame rate                                  |

|                  |                    |                                                         |
|------------------|--------------------|---------------------------------------------------------|
| IP Camera (RTSP) | Network (RTSP URL) | Set Camera Type to <i>RTSP</i> and enter the stream URL |
|------------------|--------------------|---------------------------------------------------------|

## 2 First Launch

### 2.1 Starting the application

Open a terminal, navigate to the project folder, and run:

```
cd ~/NS_runtime
./start.sh
```

`start.sh` automatically:

- Kills any leftover processes on ports 8000 and 5173
- Starts the FastAPI backend on port **8000**
- Starts the React frontend on port **5173**
- Opens the dashboard in your default browser

#### Before the first run

Make sure `setup.sh` has been run at least once, otherwise dependencies will be missing.

```
chmod +x setup.sh && ./setup.sh
```

### 2.2 Accessing the dashboard

| URL                                     | Description                                    |
|-----------------------------------------|------------------------------------------------|
| <code>http://localhost:5173</code>      | Main dashboard (camera feeds + controls)       |
| <code>http://&lt;PC-IP&gt;:5173</code>  | Access from another device on the same network |
| <code>http://localhost:8000/docs</code> | API documentation (Swagger UI)                 |

To find your PC's IP address:

```
hostname -I | awk '{print $1}'
```

### 2.3 Stopping the application

Press **Ctrl + C** in the terminal where `start.sh` is running. Both the backend and frontend will shut down cleanly.

If the terminal is unavailable, kill the processes manually:

```
lsof -ti :8000 | xargs kill -9  
lsof -ti :5173 | xargs kill -9
```



## 3 Dashboard Overview

The dashboard is the main screen. It shows live video from all active cameras and lets you control each inspection line individually.

### 3.1 Camera cards

Each configured camera appears as a card in the grid. A card shows:

- **Live video feed** — with AI bounding boxes drawn on detected defects
- **Line name** — e.g., "Line 1 — Pinhole"
- **Status badge** — Running / Stopped / Error
- **Defect counter** — total defects detected this session
- **Last defect image** — thumbnail of the most recent defect

### 3.2 Per-camera controls

| Button              | Action                                                       |
|---------------------|--------------------------------------------------------------|
| ▶ <b>Start</b>      | Begin inspection on this camera                              |
| ■ <b>Stop</b>       | Pause inspection (camera feed stays visible)                 |
| ⚙ <b>Settings</b>   | Open the line configuration modal                            |
| 📁 <b>Collection</b> | Open a popup to browse saved defect images for this line     |
| 🚫 <b>Reject Off</b> | Disable the reject actuator signal (requires admin password) |

#### Reject Off requires admin password

Disabling the reject signal is a safety-critical action. You will be prompted for the admin password before it takes effect.

### 3.3 Dashboard layout

Cards can be resized and repositioned by dragging. The layout is saved automatically and restored on the next launch. To reset the layout to defaults, go to the **Admin** page and click *Reset Layout*.

### 3.4 Collection popup

Clicking the **Collection** button opens a modal showing the most recent defect images for that camera. Images are organized by date and hour. You can scroll through them, download individual images, or clear old previews.

## 4 Line (Camera) Configuration

Each inspection line is configured through the **Settings modal** (⚙ button on each card) or from the Admin page. Changes are saved to `workers/worker-XX/config.json`.

### 4.1 Adding a new line

- 1 Go to the **Admin** page (top-right icon).
- 2 Click **Add Line**.
- 3 Fill in the Line Name (e.g., *Line 3 — Scratch*).
- 4 Configure camera, detector, and save path (see sections below).
- 5 Click **Save**. The new card appears on the dashboard.

### 4.2 Camera settings

| Field                    | Description                                                                          | Example                                                    |
|--------------------------|--------------------------------------------------------------------------------------|------------------------------------------------------------|
| <b>Camera Type</b>       | Type of camera hardware                                                              | <code>basler</code> / <code>usb</code> / <code>rtsp</code> |
| <b>Camera IP / Index</b> | For Basler: IP address. For USB: device index (0, 1, ...).<br>For RTSP: full URL.    | <code>192.168.1.101</code> or <code>0</code>               |
| <b>PFS File</b>          | Basler camera settings file exported from Pylon Viewer.<br>Leave blank for USB/RTSP. | <code>camera.pfs</code> (relative to worker directory)     |
| <b>Crop Region</b>       | Optional. Crop the frame to a rectangle: <code>x, y, w, h</code> in pixels.          | <code>100, 50, 1000, 800</code>                            |
| <b>Rotation</b>          | Rotate the frame. Values: 0, 90, 180, 270.                                           | <code>0</code>                                             |

#### Basler GigE setup tip

The camera and PC must be on the same subnet. Assign a static IP to the NIC connected to the camera:

```
sudo ip addr add 192.168.1.1/24 dev eth0
```

## 4.3 Detector type

Choose which AI model runs on each frame:

| Detector  | Best for                                                               | Model file                                    |
|-----------|------------------------------------------------------------------------|-----------------------------------------------|
| YOLO      | General defect detection — locates defect position with a bounding box | <code>.pt</code> (Ultralytics YOLOv8/v12)     |
| PaddleOCR | Text / label inspection — verifies printed text matches expected value | Built-in (no file needed unless custom model) |
| CNN       | Whole-image classification — OK vs. Defect for the entire image        | <code>.pth</code> (PyTorch)                   |

### YOLO detector settings

| Field      | Description                                                                                                |
|------------|------------------------------------------------------------------------------------------------------------|
| Model Path | Path to <code>.pt</code> weight file, relative to the worker directory. E.g., <code>weights/best.pt</code> |
| Device     | <code>cuda</code> for GPU (faster), <code>cpu</code> for CPU-only machines                                 |

### PaddleOCR detector settings

| Field         | Description                                                                                                |
|---------------|------------------------------------------------------------------------------------------------------------|
| Expected Text | The string that must appear in the image. If the recognized text does not match, it is marked as a defect. |
| Language      | OCR language code: <code>en</code> (English), <code>korean</code> , <code>ch</code> (Chinese), etc.        |
| Use GPU       | Enable GPU acceleration for OCR inference.                                                                 |

### CNN detector settings

| Field       | Description                                                                       |
|-------------|-----------------------------------------------------------------------------------|
| Model Path  | Path to <code>.pth</code> weight file, relative to the worker directory.          |
| Input Size  | Image size the model expects (e.g., <code>224</code> for 224×224).                |
| Class Names | Comma-separated class names in training order, e.g. <code>normal, defect</code> . |

## 4.4 Confidence thresholds

Thresholds control *how confident* the model must be before a detection is treated as a defect. Higher values = fewer false positives, but may miss real defects.

| Class name | Threshold | Meaning                                                            |
|------------|-----------|--------------------------------------------------------------------|
| defect     | 0.70      | Reject only if the model is $\geq 70\%$ confident                  |
| pinhole    | 0.85      | Pinhole is hard to detect — higher threshold to avoid false alarms |
| scratch    | 0.60      | Scratch: lower threshold (less margin for error)                   |

#### Tuning tip

Start with 0.70. If you see too many false rejects, increase the threshold. If real defects are being missed, decrease it. Adjust one class at a time.

## 4.5 Reject timing

| Field                 | Description                                                                                                                              |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Reject Delay (frames) | How many frames to wait before activating the reject actuator. Compensates for the distance between the camera and the reject mechanism. |
| Reject Duration (ms)  | How long the reject signal stays active (milliseconds).                                                                                  |

#### Calculating reject delay

Measure the physical distance (cm) from the camera to the reject actuator, then divide by the product travel per frame. Recalibrate if conveyor speed changes.

# 5 Save Path Configuration

**This section is critical.** Incorrect save path configuration means defect images will not be saved, or will be saved to the wrong location. Read this section carefully before starting a production line.

## 5.1 Where images are saved

Every defect image is saved under a root directory called **save\_root**. You configure `save_root` per inspection line in the line settings modal.

**Important: path is relative to the worker directory**  
All paths in `config.json` — including `save_root` — are *relative to that line's worker directory*, not to the project root. See Section 6.3 for details.

## 5.2 Setting the save path

### Option A — Using the Settings modal (recommended)

- 1 Click the **Settings** button on the camera card.
- 2 Scroll to the **Storage** section.
- 3 In the **Save Root** field, enter the path where defect images should be stored.
- 4 Click **Save**.

### Path examples

| What you type                                   | Actual save location                                                             |
|-------------------------------------------------|----------------------------------------------------------------------------------|
| <code>data</code>                               | <code>~/NS_runtime/workers/worker-01/data/</code><br>(relative — recommended)    |
| <code>/mnt/nas/line1</code>                     | <code>/mnt/nas/line1/</code><br>(absolute — NAS or external drive)               |
| <code>/home/nongshim/defect_images/line1</code> | <code>/home/nongshim/defect_images/line1/</code><br>(absolute — custom location) |

Use absolute paths for NAS / external storage

If saving to a network share or an external drive, always use the full absolute path (starting with `/`). Relative paths are only convenient when saving locally inside the worker directory.

Option B — Editing config.json directly

Open the worker's config file in a text editor:

```
nano ~/NS_runtime/workers/worker-01/config.json
```

Find and edit the `save_root` field:

```
{
  "save_root": "/mnt/nas/inspection/line1",
  "save_normal": false,
  "preview_count": 20
}
```

5.3 Saved folder structure

Once a `save_root` is configured, the system automatically creates this directory structure:

```
save_root/                                ← your configured path
├── defect/
│   ├── 2026-05-23/                       ← date folder (auto-created)
│   │   ├── 14/                           ← hour folder (auto-created)
│   │   │   ├── defect-0.93-20260523-140032-line1.jpg ← defect image
│   │   │   └── defect-0.93-20260523-140032-line1.txt ← detection coordinates
├── preview/                             ← most recent N defect images (quick view)
├── archive/                             ← images that overflowed preview/ are moved here
└── normal/                             ← only created when save_normal = true
```

File naming format

Each defect image is named:

```
defect-{confidence}-{YYYYMMDD}-{HHmmss}-{line_name}.jpg
```

| Part         | Meaning             | Example    |
|--------------|---------------------|------------|
| defect       | Fixed prefix        | defect     |
| {confidence} | AI confidence score | 0.93 = 93% |
| {YYYYMMDD}   | Date                | 20260523   |
| {HHmmss}     | Time                | 140032     |

`{line_name}`

The line name from config

`line1`

## Preview and archive

| Folder                | Purpose                                                                                    | Controlled by                                      |
|-----------------------|--------------------------------------------------------------------------------------------|----------------------------------------------------|
| <code>preview/</code> | Holds the most recent N defect images for quick viewing in the dashboard Collection popup. | <code>preview_count</code> in config (default: 20) |
| <code>archive/</code> | When <code>preview/</code> exceeds the limit, older images are moved here automatically.   | Automatic                                          |
| <code>normal/</code>  | Saves images of non-defective products. Disabled by default (saves a lot of disk space).   | <code>save_normal</code> in config                 |

### Disk space warning

With `save_normal = true` at high frame rates, disk usage can be very high. Keep `save_normal = false` unless you specifically need to collect normal images for retraining the model.

## 5.4 AWS S3 remote storage

Defect images can be automatically uploaded to an S3 bucket in addition to local storage. Configure S3 from the **Admin** page → **S3 Settings**.

| Field                    | Description                                                        |
|--------------------------|--------------------------------------------------------------------|
| <b>Bucket Name</b>       | Your S3 bucket name                                                |
| <b>Region</b>            | AWS region, e.g. <code>us-east-1</code>                            |
| <b>Access Key ID</b>     | AWS IAM access key with S3 write permission                        |
| <b>Secret Access Key</b> | Corresponding secret key                                           |
| <b>S3 Prefix</b>         | Optional folder prefix inside the bucket, e.g. <code>line1/</code> |

### S3 upload is non-blocking

Images are always saved locally first. S3 upload happens in the background and does not affect inspection performance. If the upload fails, images remain on disk.



## 6 Settings Files Reference

### 6.1 config.json (per-line)

Located at `workers/worker-XX/config.json`. One file per inspection line. Edited automatically when you save from the Settings modal, but can also be edited by hand.

#### Full annotated example

```
{
  // — Identity —————
  "line_name":      "Line 1 - Pinhole",
  "worker_id":      "worker-01",

  // — Camera —————
  "camera_type":    "basler",      // basler | usb | rtsp
  "camera_ip":      "192.168.1.101",
  "pfs_file":       "camera.pfs",  // relative to worker dir
  "crop":           null,          // or [x, y, w, h]
  "rotate":         0,             // 0 | 90 | 180 | 270

  // — Product / Detector —————
  "detector_type":   "yolo",        // yolo | paddleocr | cnn
  "model_path":      "weights/best.pt",
  "device":          "cuda",        // cuda | cpu
  "class_thresholds": {
    "defect":        0.70,
    "pinhole":       0.85
  },
  "detector_config": {},           // extra settings per detector type

  // — Reject Timing —————
  "reject_delay_frames": 12,        // frames between detect and actuate
  "reject_duration_ms": 300,        // how long the signal stays HIGH
  "reject_enabled":    true,

  // — Save Path —————
  "save_root":       "data",        // relative to worker dir – or absolute
  "save_normal":     false,         // save non-defect frames? (uses disk)
  "preview_count":   20             // max images in preview/ folder
}
```

#### Field reference

| Field                            | Type         | Required    | Notes                                                                       |
|----------------------------------|--------------|-------------|-----------------------------------------------------------------------------|
| <code>line_name</code>           | string       | Yes         | Display name shown on the dashboard card                                    |
| <code>camera_type</code>         | string       | Yes         | <code>basler</code> / <code>usb</code> / <code>rtsp</code>                  |
| <code>camera_ip</code>           | string       | Basler only | IP address of the Basler camera                                             |
| <code>pfs_file</code>            | string       | Basler only | Camera settings file, relative to worker dir                                |
| <code>crop</code>                | array / null | No          | <code>[x, y, w, h]</code> in pixels; <code>null</code> = full frame         |
| <code>rotate</code>              | int          | No          | Degrees: 0, 90, 180, or 270                                                 |
| <code>detector_type</code>       | string       | Yes         | <code>yolo</code> / <code>paddleocr</code> / <code>cnn</code>               |
| <code>model_path</code>          | string       | YOLO / CNN  | Path to weight file, relative to worker dir                                 |
| <code>device</code>              | string       | No          | <code>cuda</code> or <code>cpu</code> (default: <code>cuda</code> )         |
| <code>class_thresholds</code>    | object       | No          | Map of class name → confidence float (0.0–1.0)                              |
| <code>detector_config</code>     | object       | No          | Detector-specific extra settings (e.g., <code>expected_text</code> for OCR) |
| <code>reject_delay_frames</code> | int          | No          | Default: 0                                                                  |
| <code>reject_duration_ms</code>  | int          | No          | Default: 300 ms                                                             |
| <code>reject_enabled</code>      | bool         | No          | Default: true                                                               |
| <code>save_root</code>           | string       | Yes         | Base folder for all saved images                                            |
| <code>save_normal</code>         | bool         | No          | Default: false                                                              |
| <code>preview_count</code>       | int          | No          | Default: 20                                                                 |

## 6.2 global\_settings.json

Located at `configs/global_settings.json`. Controls system-wide settings shared across all lines.

```
{
  // — Admin —————
  "admin_password":    "1",           // CHANGE THIS IMMEDIATELY

  // — Dashboard —————
  "grid_layout":      [],             // auto-managed by the UI
  "max_workers":      10,             // maximum number of camera lines

  // — S3 (optional) —————
  "s3_enabled":       false,
```

```
"s3_bucket":      "",
"s3_region":      "us-east-1",
"s3_access_key":   "",
"s3_secret_key":   "",
"s3_prefix":      ""
}
```

### Change the admin password before production use

The default password is `1` . Change it to something secure via the Admin page or by editing `admin_password` in this file directly.

## 6.3 Path conventions

**Core rule: paths in config.json are relative to the worker directory**

The worker directory for worker-01 is:

```
~/NS_runtime/workers/worker-01/
```

So `"save_root": "data"` maps to:

```
~/NS_runtime/workers/worker-01/data/
```

And `"model_path": "weights/best.pt"` maps to:

```
~/NS_runtime/workers/worker-01/weights/best.pt
```

### When to use absolute paths

| Scenario                                   | Recommended path                                |
|--------------------------------------------|-------------------------------------------------|
| Saving locally inside the worker directory | Relative: <code>data</code>                     |
| Saving to a NAS or network share           | Absolute: <code>/mnt/nas/line1</code>           |
| Saving to an external USB drive            | Absolute: <code>/media/usb/defects/line1</code> |
| Shared folder accessible by multiple PCs   | Absolute: <code>/home/shared/inspection</code>  |

### Mounting a NAS for remote storage

```
# Install NFS client
sudo apt install -y nfs-common

# Mount the NAS share
```

```
sudo mount 192.168.1.200:/exports/inspection /mnt/nas

# Auto-mount on boot (add to /etc/fstab)
192.168.1.200:/exports/inspection /mnt/nas nfs defaults 0 0
```

Then set `save_root` to `/mnt/nas/line1` in config.json.

## 7 Admin Page

### 7.1 Accessing admin

- 1 Click the **Admin** icon (gear/lock icon) in the top-right corner of the dashboard.
- 2 Enter the admin password (default: **1** ).
- 3 The Admin page opens.

### 7.2 Available actions

| Action                | Description                                              |
|-----------------------|----------------------------------------------------------|
| Add Line              | Create a new inspection line / camera card               |
| Delete Line           | Remove a line and its card from the dashboard            |
| Reset Layout          | Restore dashboard grid to default positions              |
| Change Admin Password | Update the admin password stored in global_settings.json |
| S3 Settings           | Configure AWS S3 bucket for remote image upload          |
| View History          | Browse the defect history log (count per line per day)   |

### 7.3 Changing the admin password

- 1 Open the Admin page.
- 2 Click **Change Password**.
- 3 Enter current password, then new password twice.
- 4 Click **Save**.

Alternatively, edit `configs/global_settings.json` directly:

```
{  
  "admin_password": "your-secure-password"
```

}

## 7.4 S3 configuration

- 1 On the Admin page, click **S3 Settings**.
- 2 Toggle **Enable S3 Upload**.
- 3 Fill in Bucket Name, Region, Access Key ID, Secret Access Key.
- 4 Optionally set a Prefix (folder path inside the bucket).
- 5 Click **Test Connection** to verify credentials.
- 6 Click **Save**.

### IAM permissions required for S3

The AWS IAM user must have at minimum: `s3:PutObject` and `s3:GetBucketLocation` on the target bucket.

## 8 Troubleshooting

### 8.1 Application won't start

| Symptom                            | Likely cause              | Fix                                                                                          |
|------------------------------------|---------------------------|----------------------------------------------------------------------------------------------|
| <code>uv: command not found</code> | uv not in PATH            | <pre>echo 'export PATH="\$HOME/.local/bin:\$PATH"' &gt;&gt; ~/.bashrc source ~/.bashrc</pre> |
| Port 8000 / 5173 already in use    | Old process still running | <pre>lsof -ti :8000   xargs kill -9 lsof -ti :5173   xargs kill -9</pre>                     |
| <code>ModuleNotFoundError</code>   | Python deps not installed | <pre>cd ~/NS_runtime &amp;&amp; uv sync</pre>                                                |

### 8.2 Camera not connecting

| Camera type        | Check                                                                                                                                                                                                                                                                                |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Basler GigE</b> | <ol style="list-style-type: none"> <li>1. Verify the camera and PC NIC are on the same subnet.</li> <li>2. Check the IP in config.json matches the camera's IP (visible in Pylon Viewer).</li> <li>3. Ensure <code>camera.pfs</code> file exists in the worker directory.</li> </ol> |
| <b>USB Webcam</b>  | Try index 0, 1, 2... until the feed appears. Check with: <code>ls /dev/video*</code>                                                                                                                                                                                                 |
| <b>RTSP</b>        | Test the stream URL with: <pre>ffplay rtsp://192.168.1.100:554/stream</pre>                                                                                                                                                                                                          |

### 8.3 Images not being saved

| Check                                                   | How                                                 |
|---------------------------------------------------------|-----------------------------------------------------|
| Is <code>save_root</code> set correctly in config.json? | Open Settings modal and verify the Save Root field. |

|                                              |                                                                  |
|----------------------------------------------|------------------------------------------------------------------|
| Does the directory exist and is it writable? | <pre>ls -la /your/save/path touch /your/save/path/test.txt</pre> |
| Is the inspection worker running?            | Status badge on camera card should show <i>Running</i> .         |
| Are thresholds too high?                     | Lower confidence thresholds temporarily to test saving.          |

## 8.4 GPU not detected

```
# Verify NVIDIA driver
nvidia-smi

# Verify PyTorch CUDA support
uv run python -c "import torch; print('CUDA:', torch.cuda.is_available())"

# Reinstall CUDA-enabled PyTorch if needed
uv pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu118
```

## 8.5 Frontend not loading

```
cd ~/NS_runtime/frontend
rm -rf node_modules package-lock.json
npm install
npm run build
```

## 8.6 Defect detection seems wrong (too many / too few)

| Problem                                           | Action                                                                                     |
|---------------------------------------------------|--------------------------------------------------------------------------------------------|
| Too many false positives (good products rejected) | Increase the class threshold (e.g., 0.70 → 0.85).                                          |
| Too many false negatives (defects not caught)     | Decrease the class threshold (e.g., 0.70 → 0.55).                                          |
| Wrong class being detected                        | Verify <code>class_thresholds</code> key names match the model's class names exactly.      |
| Model not loading                                 | Check that <code>model_path</code> points to an existing file inside the worker directory. |



